

3 IMPLEMENTATION

We have implemented a prototype of the described parallel compiler in a mix of C++ and Futhark [6] code. The former is used for the pre-processing tools to generate the pre-computed tables required for the lexical analyzer and parser, and for the compiler's driver code. The driver code loads these tables onto the GPU at runtime, along with the source code to compile in verbatim, after which it launches the compilation passes. All of the compilation passes are written in the Futhark functional programming language.

3.1 Lexical Analyzer & Parser

The pre-processing for the lexical analyzer and parser is combined into a single tool, which generates the appropriate data structures during the build process of the compiler. For the lexical analyzer, a single deterministic finite automaton that is run once for the entire input is constructed from a lexical grammar provided as text file. It is encoded into three (table) data structures to support simulation of the automaton using the algorithm of Hillis & Steele. To reduce the memory space required, we compute all reachable compositions of unary transition functions ahead of time and assign an (integer) identifier to each one of them.

To generate the parser, a syntactical grammar must be supplied in the form of a text file containing a number of productions. This input is processed into a total of three data structures. The generation algorithm that is employed is a straightforward implementation of the method discussed in [20]. As our compiler is designed around a $LLP(1, 1)$ grammar, we can pre-compute the stack configuration changes for every possible pair of token types and store this in a two-dimensional lookup table.

At runtime, the lexical analyzer automaton is evaluated using the algorithm of Hillis & Steele, implemented using parallel map for table lookups and a parallel prefix scan to deduce all token types that have been emitted during the evaluation. This sequence of token types is the final token stream, from which the parser will build the parse tree. The parallel parsing algorithm that we consider first maps each subsequent pair of symbols to stack configuration changes using the lookup table and a parallel string extraction algorithm. Secondly, the validity of the input is determined by verifying that the concatenated stack configuration changes are balanced using a parallel bracket matching algorithm [1]. Finally, the final parse can be computed in a similar fashion, but using a different lookup table and superstring. This is converted to a parse tree using a variant of depth-first traversal implemented using parallel primitives.

3.2 Semantic Analysis

This stage is implemented as a series of passes that analyze and manipulate the parse tree. This consists of passes such as extracting lexemes to literals, variable resolution, function resolution and type analysis. The implementation of these passes is facilitated by a set of common operations on the inverted tree data structure. For example, a typical operation is to find the root of certain subtrees. This operation can be implemented in parallel by applying the algorithm by Hillis & Steele [9] to find the end of a linked list on each of the nodes simultaneously. When the tree is converted to an AST, the inverted tree is converted from pre-order to post-order

and a set of arrays is created where each array stores a particular node property (e.g., data type, parent index, depth of the node in the AST, literals). So, all information regarding a node is now easily reachable without tree walks, which simplifies the implementation of the subsequent code generation stages.

3.3 Intermediate Code Generation

First, the number of instructions to be generated is computed¹ and an instruction buffer is allocated. Second, a mapping is computed from nodes in the tree to instruction locations in the buffer. Both rely on a parallel map to map each node type to the number of instructions required to encode it. Small adjustments are made for special cases (e.g., branch nodes) and a function offset table, containing start and size of each function, is computed. To generate the actual instructions, child nodes must be processed before parent nodes. As such, a bottom-up tree walk is performed which will process all nodes at the same depth in parallel. By assuming each node will yield at most four instructions, we spawn four threads for each AST node. Each of these threads will determine instruction opcode, registers and branch target according to the node type.

3.4 Optimization

Simple optimizations can be implemented that amend instructions in the intermediate representation or mark instructions for removal in the instruction mask. As an example, consider dead expression removal, which removes any expression of which the result is not used. To do so, we repeatedly update a table that maps virtual register numbers to a boolean indicating whether that register is used, using parallel map and scatter operations. For instructions of which the destination register is not used, the booleans for the source registers are set to false. Because of the correspondence between virtual register numbers and instruction location, we can directly update the instruction mask from this table to indicate the instructions that can be optimized away. Instructions that induce side-effects are never removed.

3.5 Machine Code Generation

First, the physical register numbers to be encoded into the instructions must be determined. This register allocation starts with a lifetime analysis to determine which registers are in use at which point of the program. This analysis is performed in parallel for each function (and cannot be parallelized at the level of instructions). Each instruction of a function is visited sequentially and a lifetime mask, which contains a bit indicating availability of every physical register, is updated according to the operands. Physical registers are greedily assigned by finding a free register in the lifetime mask. If no free register is available, a register will be selected from a statically defined set of suitable scratch registers and marked as a spilled register. The assignment of virtual to physical registers is stored in the virtual register table together with a swap bit that identifies if that register needs to be spilled at any point.

After lifetime analysis has completed, stack space is allocated by computing a stack offset table that maps virtual registers to integer stack offsets. This computation is performed for all functions in parallel using the swap bits from the virtual register table.

¹Note that a single node in the AST may result in more than one instruction.